Práctica 2: Gestión de Representación Deportiva

Programación 2

Curso 2025-2026

En esta segunda práctica de la asignatura desarrollarás un gestor de jugadores de baloncesto para una agencia de representación deportiva. Los conceptos necesarios para desarrollar esta práctica se trabajan en el *Tema 1*, *Tema 2* y *Tema 3* de teoría.

Condiciones de entrega

- La fecha límite de entrega para esta práctica es el **viernes 27 de marzo**, hasta las **23:59**.
- Debes entregar un fichero llamado `prac2.cc` con el código de todas las funciones.
- Debes entregar también un fichero `estilo.txt` que contendrá el enlace a la revisión de estilo hecha por ChatGPT tal y como se describe en la Sección 2.

Código de honor

Si se detecta copia (total o parcial) en tu práctica, tendrás un **0** en la entrega y se informará a la dirección de la Escuela Politécnica Superior para que adopte medidas disciplinarias

Está bien discutir con tus compañeros posibles soluciones a las prácticas.
Está bien apuntarte a una academia si sirve para obligarte a estudiar y hacer las prácticas.

Está mal copiar código de otros compañeros o pedirle a ChatGPT que te haga la práctica.
Está mal apuntarte a una academia para que te hagan las prácticas.

Si necesitas ayuda acude a tu profesor/a.
No copies.

Normas generales

- Debes entregar la práctica exclusivamente a través del servidor de prácticas del Departamento de Lenguajes y Sistemas Informáticos (DLSI): `https://pracdlsi.dlsi.ua.es`
- Cuestiones que debes tener en cuenta al hacer la entrega:
  - El usuario y la contraseña para entregar prácticas son los mismos que utilizas en UACloud.
  - Puedes entregar la práctica varias veces, pero sólo se corregirá la última entrega.
  - No se admitirán entregas por otros medios, como el correo electrónico o UACloud.
  - No se admitirán entregas fuera de plazo.
- Tu práctica debe poder ser compilada sin errores con el compilador de C++ existente en la distribución de Linux de los laboratorios de prácticas.

Desarrollado por @prog2ua y publicado bajo licencia
Creative Commons Reconocimiento 4.0 Internacional

- Si tu práctica no se puede compilar, su calificación será 0.
- Si tu práctica utiliza instrucciones que no se han visto en el *Tema 1*, *Tema 2* o *Tema 3*, su calificación será 0.
- El 90 % de la nota de la práctica dependerá de la corrección automática, por lo que es imprescindible que respetes estrictamente los textos y los formatos de salida que se indican en este enunciado. El otro 10 % dependerá de la revisión de código que haga ChatGPT, por lo que debes también ajustarte a la guía de estilo de la asignatura.
- Al comienzo de todos los ficheros fuente entregados debes incluir un comentario con tu NIF (o equivalente) y tu nombre. Por ejemplo:

prac2.cc

```
// DNI 12345678X GARCIA GARCIA, JUAN MANUEL
...
```

- El cálculo de la nota de la práctica y su relevancia en la nota final de la asignatura se detallan en las diapositivas de presentación de la asignatura (*Tema 0*).

1. Descripción de la práctica

La agencia deportiva "ProSports" quiere gestionar la información de los jugadores de baloncesto que representa, así como llevar un seguimiento de sus valoraciones en cada partido para poder crear rankings de rendimiento. Tu labor es implementar un programa que gestione los jugadores, permita añadir valoraciones de sus actuaciones en cada partido y un ranking. El programa permitirá añadir y borrar jugadores, añadir valoraciones a los jugadores y generar informes. También habrá opciones para importar y exportar los datos de la agencia y sus jugadores a distintos formatos usando ficheros de texto y binarios.

2. Detalles de implementación

En Moodle se publicarán varios ficheros que necesitarás para la correcta realización de la práctica:

- `prac2.cc`. Este fichero contiene un esqueleto de programa sobre el que realizar tu práctica. Descárgalo y añade tu código en él. Este fichero contiene la siguiente información:
  - Los registros (struct) necesarios para hacer la práctica.
  - Un tipo enumerado `Error` que contiene todas las posibles clases de error que se pueden dar en esta práctica.
  - Una función `error()` que se encarga de mostrar el correspondiente mensaje de error por pantalla en función del parámetro que se le pase.
  - Una función `showMainMenu()` que muestra por pantalla el menú principal.
  - Una función `main()` que implementa la gestión del menú principal y llama a las funciones correspondientes dependiendo de la opción elegida por el usuario.
- `autocorrector-prac2.tgz`. Contiene los ficheros del autocorrector para probar la práctica con algunas pruebas de entrada. La corrección automática de la práctica se realizará con un programa similar, con estas pruebas y otras más definidas por el profesorado de la asignatura.
- `prac2`. Fichero ejecutable de la práctica (compilado para máquinas Linux de 64 bits) desarrollado por el profesorado de la asignatura, para que puedas probarlo con las entradas que quieras y ver la salida correcta esperada.

Desarrollado por @prog2ua y publicado bajo licencia
Creative Commons Reconocimiento 4.0 Internacional

El código de tu práctica debe seguir las indicaciones establecidas en la *Guía de estilo y buenas prácticas de programación en C++* que hay disponible en Moodle. Para comprobar que es así, se ha habilitado un GPT personalizado con el que analizar tu código y determinar si efectivamente sigue la guía de estilo:

https://chatgpt.com/g/g-696b5bb084ec8191a4682865e186d34f-profe-3000

Sigue las instrucciones que te proporciona el propio *chatbot*. Una vez que éste te diga que tu código siga perfectamente el estilo de la asignatura, copia el enlace al chat (opción *Compartir* en la esquina superior derecha de ChatGPT) en un fichero llamado `estilo.txt` y entrégalo junto con el fichero `prac2.cc` en el servidor de prácticas.

- En la corrección de la práctica, si se pide al usuario que introduzca una opción del menú, se probará siempre con un valor tipo `char`, que podría ser `1` o `z`, pero nunca se introducirá un valor de otro tipo (cadena de caracteres, número real, ...).
- Debes crear las funciones que consideres oportunas para que tu código sea más sencillo y fácil de leer e interpretar por un humano. No puedes, por ejemplo, meter todo tu código en el `main`. Al menos, deberás declarar funciones para cada una de las opciones del menú principal y del menú de ficheros.
- El resto de decisiones en la implementación de la práctica quedan a tu criterio.

3. Componentes

El programa a desarrollar debe poder gestionar dos elementos fundamentales: los jugadores (`Player`), que almacenarán la información sobre todos los deportistas representados por la agencia, y la agencia (`Agency`), que almacenará los jugadores en un vector. Los siguientes apartados describen la estructura de cada uno de estos componentes en detalle.

Además de los registros `Player` y `Agency`, hay dos registros adicionales `BinPlayer` y `BinAgency` para poder almacenar información y trabajar con ficheros binarios. Esto último es necesario porque los dos primeros registros contienen campos de tipo `string` y `vector` que, como sabes, no pueden utilizarse para leer o escribir información de ficheros binarios al no tener un tamaño fijo (si no sabes de qué estoy hablando, consulta la diapositiva 33 del Tema 3 de teoría).

Los registros `BinPlayer` y `BinAgency` sólo se utilizarán cuando haya que leer o escribir en un fichero binario. Para el resto de funcionalidades de la práctica se usarán los registros `Player` y `Agency`.

3.1. Player

La estructura `Player` almacena información de un jugador dentro de la agencia. Cada jugador contendrá un identificador único (`id`), nombre completo (`name`), el nombre del equipo al que pertenece (`team`), número de dorsal (`dorsal`), su posición en el campo (`position`) y la lista de valoraciones obtenidas en los partidos jugados (`ratings`).

```
struct Player {
    unsigned int id;
    string name;
    string team;
    unsigned int dorsal;
    unsigned int position;
    vector<int> ratings;
};
```

El campo `id` es un identificador único asignado automáticamente. El campo `name` debe contener tanto el nombre como el apellido del jugador. El campo `team` contiene el nombre del equipo al que pertenece el jugador. El `dorsal` es el número que lleva el jugador en su camiseta (entre 0 y 99). La posición indica el rol del jugador en el campo (valores del 1 al 5, donde cada número representa una posición específica: 1-*Point*

Desarrollado por @prog2ua y publicado bajo licencia
Creative Commons Reconocimiento 4.0 Internacional

Guard, 2-*Shooting Guard*, 3-*Small Forward*, 4-*Power Forward*, 5-*Center*). El vector `ratings` almacena las valoraciones que ha obtenido el jugador en cada partido que ha disputado (valores entre -50 y 50).

3.2. BinPlayer

Este tipo de registro se utilizará para leer y escribir información de un jugador en el fichero binario:

```
struct BinPlayer {
    unsigned int id;
    char name[kMAXSTRING];
    char team[kMAXSTRING];
    unsigned int dorsal;
    unsigned int position;
    unsigned int numRatings;
};
```

En este registro se han introducido dos cambios con respecto al registro `Player`:

- Los campos `string` se sustituyen por cadenas de caracteres, donde `kMAXSTRING` es una constante de tipo entero cuyo valor es 50.
- Se ha añadido el campo `numRatings` que indica cuántas valoraciones tiene el jugador. Este campo es necesario porque la lista de valoraciones se guardará después del registro `BinPlayer` en el fichero binario, tal y como se describe en la Sección 5.6.3.

3.3. Agency

Este registro representa la agencia de representación y contiene un nombre (`name`), un vector de jugadores (`players`) y un campo entero `nextId` que almacenará el identificador que deberá asignarse al siguiente jugador que se cree.

```
struct Agency {
    string name;
    vector<Player> players;
    unsigned int nextId;
};
```

El campo `nextId` tendrá el valor 1 al inicio del programa y se incrementará de uno en uno para cada nuevo jugador. Es decir, el primer jugador que se cree tendrá como identificador el 1, el segundo tendrá el 2 y así sucesivamente.

En el programa principal (`main`) deberás declarar una variable de tipo `Agency` y asignar a su campo `name` el valor "ProSports Agency" y a su campo `nextId` el valor 1. El vector `players` estará inicialmente vacío e irá creciendo conforme se añadan nuevos jugadores al programa. Tu programa gestionará una única agencia, que contendrá todos los jugadores.

3.4. BinAgency

Este tipo de registro se utilizará para leer y escribir información de la agencia en el fichero binario:

```
struct BinAgency {
    char name[kMAXSTRING];
    unsigned int nextId;
};
```

Igual que sucedía con `BinPlayer`, el campo `string name` se sustituye por el campo `char name[kMAXSTRING]`. También ha desaparecido el vector de jugadores.

Desarrollado por @prog2ua y publicado bajo licencia
Creative Commons Reconocimiento 4.0 Internacional

4. Menú

Al ejecutar la práctica se mostrará de inicio por pantalla el menú principal del programa, quedando a la espera de que el usuario elija una opción:

Terminal

```
[Menu]
1- Show players
2- Add player
3- Add player rating
4- Delete player
5- Show ranking
6- Import/Export
q- Quit
Option:
```

Este menú será el que aparezca por pantalla cuando el usuario inicie el programa y por lo tanto deberá de gestionarse en la función `main`. Las opciones válidas que puede introducir el usuario son los números del `1` al `6` y la letra `q`. Si la opción elegida no es ninguna de éstas (por ejemplo un `8`, una `x` o un `;`) se emitirá el error `ERR_OPTION` llamando a la función `error` con dicho parámetro. Cuando el usuario elige una opción correcta, se debe ejecutar el código asociado a dicha opción. Al finalizar, volverá a mostrar el menú principal y a pedir otra opción, hasta que el usuario decida salir del programa con la opción `q`.

5. Opciones

En los siguientes apartados se describe el funcionamiento que deberá tener cada una de las opciones que se muestran en el menú principal del programa.

5.1. Show players

Esta funcionalidad se activa con la opción 1 del menú y mostrará un listado con la información de todos los jugadores y sus valoraciones. Para cada jugador se mostrará su información básica seguida de una tabla con sus valoraciones por partido, tal y como se muestra en el siguiente ejemplo:

Terminal

```
Id: 1
Name: Kevin Love
Team: Utah Jazz
Dorsal: 11
Position: Power Forward
Game | Rating
1 | -5

Id: 2
Name: Facundo Campazzo
Team: Real Madrid
Dorsal: 7
Position: Point Guard
Game | Rating
1 | 35
2 | 28

Id: 3
Name: Nikola Jokic
Team: Denver Nuggets
Dorsal: 15
Position: Center
```

Desarrollado por @prog2ua y publicado bajo licencia
Creative Commons Reconocimiento 4.0 Internacional

En este ejemplo, el jugador con identificador 1 tiene una lista de una valoración correspondientes a un partido jugado. El número de partido (`Game`) es simplemente la posición en el vector (1 para el primer elemento, 2 para el segundo, etc.). El jugador con identificador 2 ha jugado dos partidos. El tercer jugador no tiene ninguna valoración registrada, por lo que se mostrará su información básica pero no aparecerá la tabla de `Game/Rating`. En la tabla `Game/Rating` se usará el carácter `'|'` para separar las columnas.

5.2. Add player

Esta opción permite añadir un jugador nuevo al programa. Se activa cuando el usuario elige la opción 2 del menú principal. En ella se pedirá al usuario que introduzca el nombre del jugador mostrando el siguiente mensaje:

Terminal
```
Enter player name:
```

El valor introducido por el usuario deberá revisarse para comprobar que sea un nombre válido. Se considerarán válidos los nombres que cumplan las siguientes condiciones:

- Solo pueden contener letras (ni números, ni signos de puntuación)
- Siempre contendrán nombre y apellido separados por un único espacio en blanco. Por ejemplo: `Victor Wembanyama`, `Karl Anthony` o `Abdul Jabbar`.
- Tanto el nombre como el apellido deben contener al menos 2 caracteres.
- Tanto el nombre como el apellido tendrán la primera letra en mayúscula y las demás en minúsculas.

En la corrección no se introducirán caracteres acentuados, ni cualquier otro que no pertenezca al alfabeto inglés.

Si se introduce un nombre inválido o una cadena vacía, se emitirá el error `ERR_NAME` y se volverá a pedir al usuario que introduzca el nombre mostrando el mismo mensaje.

También será necesario comprobar si ya existe un jugador en la agencia con ese nombre. En ese caso se mostrará el error `ERR_NAME_EXISTS` y se volverá a pedir al usuario que introduzca el nombre con el mismo mensaje.

Si el nombre es correcto y no está repetido, se asignará al jugador y se pedirá a continuación al usuario que introduzca el nombre del equipo al que pertenece mostrando el siguiente mensaje:

Terminal
```
Enter team name:
```

El nombre del equipo debe cumplir las mismas condiciones de validación que el nombre del jugador. Por ejemplo: `Real Madrid`, `Fc Barcelona` o `Chicago Bulls`. Si el nombre del equipo no es válido o se introduce una cadena vacía, se emitirá el error `ERR_NAME` y se volverá a pedir al usuario que introduzca el nombre del equipo mostrando el mismo mensaje. En este caso no se comprueba si el nombre del equipo ya existe, ya que pueden haber múltiples jugadores que pertenezcan al mismo equipo.

Una vez validado el nombre del equipo, se pedirá al usuario que introduzca el dorsal mostrando el siguiente mensaje:

Terminal
```
Enter dorsal:
```

Desarrollado por @prog2ua y publicado bajo licencia
Creative Commons Reconocimiento 4.0 Internacional

El dorsal deberá ser un número entre 0 y 99 (ambos incluidos). Si el usuario introduce un valor no válido se mostrará el error `ERR_DORSAL` y se volverá a pedir al usuario que introduzca el dorsal mostrando el mismo mensaje.

A continuación se pedirá al usuario que introduzca la posición:

Terminal
```
Enter position:
```

La posición deberá ser un número entre 1 y 5 (ambos incluidos). Si el usuario introduce un valor no válido se mostrará el error `ERR_POSITION` y se volverá a pedir la posición mostrando el mismo mensaje.

Una vez leídos el nombre, equipo, dorsal y posición correctos se creará el jugador con el vector `ratings` vacío (sin valoraciones). Se asignará de manera automática al nuevo jugador un identificador único que se almacenará en el campo `id` de `Player`. Este identificador vendrá dado por el valor que haya en ese momento almacenado en el campo `nextId` de la estructura `Agency` que contiene toda la información del programa. Como se indicó en la Sección 3.3, este identificador único comenzará con el valor 1 y se incrementará de uno en uno para cada nuevo jugador. Por lo tanto, cada vez que añadas un jugador, deberás de incrementar el campo `nextId` para que al siguiente jugador se le asigne un nuevo identificador correcto. Tras asignar el identificador, se incorporará el nuevo jugador al vector `players` del registro `Agency` y se volverá al menú principal.

Si se borra un jugador no se reutilizará su identificador, ni aunque sea el último jugador creado. Es decir, si tienes tres jugadores, con identificadores 1, 2 y 3, y se borra el 3, al crear un nuevo jugador se le asignará el identificador 4 aunque ya no exista un jugador con el identificador 3.

5.3. Add player rating

Esta opción permite añadir valoraciones de partidos a un jugador ya existente. Se activa cuando el usuario elige la opción 3 del menú principal. En ella se pedirá el identificador del jugador con el mensaje:

Terminal
```
Enter player id:
```

Si no existe un jugador con ese identificador, o se introduce la cadena vacía, se emitirá el error `ERR_PLAYER_NOT_EXISTS` y se volverá al menú principal. Si el identificador es correcto, se pedirá que se introduzcan las valoraciones con el mensaje:

Terminal
```
Enter ratings (comma-separated):
```

El usuario deberá introducir una o más valoraciones separadas por comas. Por ejemplo:

- `25,-10,18,30` (cuatro valoraciones)
- `15` (una valoración)
- `20,25,18` (tres valoraciones)

Cada valoración debe ser un número entero entre -50 y 50 (ambos incluidos). El formato debe ser números separados **únicamente por comas**, sin espacios ni otros caracteres. Si el formato no es correcto (por ejemplo, hay espacios, las valoraciones están separadas por otro carácter o se introduce la cadena vacía), se mostrará el error `ERR_RATING_FORMAT` y se volverán a pedir las valoraciones mostrando el mismo mensaje.

Desarrollado por @prog2ua y publicado bajo licencia
Creative Commons Reconocimiento 4.0 Internacional

Por otra parte, si alguna de las valoraciones introducidas no está en el rango válido (-50 a 50), se mostrará el error `ERR_RATING` y se volverán a pedir las valoraciones.

Una vez validadas todas las valoraciones, se añadirán al final del vector `ratings` del jugador **en el orden en que aparecen** y se volverá al menú principal.

> Las valoraciones se van acumulando en el vector. Si un jugador ya tiene 3 valoraciones y se le añaden 2 más, acabará con 5 valoraciones en total.

5.4. Delete player

Esta opción permite borrar un jugador existente en la agencia. Se activa cuando el usuario elige la opción 4 del menú principal. En ella se pedirá el identificador del jugador con el mensaje:

Terminal
```
Enter player id:
```

Si no existe un jugador con ese identificador, o se introduce la cadena vacía, se emitirá el error `ERR_PLAYER_NOT_EXISTS` y se volverá al menú principal. Si el identificador es correcto, se eliminará el jugador del vector `players` del registro `Agency` y se volverá al menú principal.

5.5. Show ranking

Esta opción permite generar un ranking con todos los jugadores ordenados por su valoración media. Se activa cuando el usuario elige la opción 5 del menú principal. Se mostrará un ranking con todos los jugadores que tengan al menos una valoración registrada, ordenados de mayor a menor valoración media.

La valoración media de un jugador se calcula como:

$$\text{Valoración media} = \frac{\text{suma de todas las valoraciones}}{\text{número de valoraciones}}$$

Un ejemplo de salida es el siguiente:

Terminal
```
Ranking | Id | Name | Dorsal | Position | Rating
1 | 2 | Facundo Campazzo | 7 | Point Guard | 31.5
2 | 1 | Kevin Love | 11 | Power Forward | 9.6
```

Los campos están separados por el carácter `'|'`. La posición en el ranking (`Ranking`) empieza en 1 para el mejor jugador y continúa de manera consecutiva con los siguientes. Si no hubiera jugadores con valoraciones registradas, se mostraría el error `ERR_NO_PLAYERS_WITH_RATINGS` y se volverá al menú principal sin mostrar nada más.

5.6. Import/Export

Esta opción permite leer la información de los jugadores desde ficheros en distintos formatos, así como almacenarla en estos mismos formatos. En esta opción se mostrará el menú de manejo de ficheros y se llamará a las distintas funciones para cada una de las opciones, de manera análoga a como se hace en el `main` con el menú principal. Esta funcionalidad se activa cuando el usuario elige la opción 6 del menú principal, tras lo cual se deberá mostrar el menú de manejo de ficheros:

Desarrollado por @prog2ua y publicado bajo licencia
Creative Commons Reconocimiento 4.0 Internacional

```
Terminal
[Import/Export]
1- Import from CSV
2- Export to CSV
3- Load data
4- Save data
b- Back to main menu
Option:
```

Las opciones válidas que puede introducir el usuario son los números del 1 al 4 y la letra b. Si la opción elegida no es ninguna de éstas (por ejemplo un 7, una x o un ;) se emitirá el error ERR_OPTION llamando a la función error y se volverá a mostrar este mismo menú. Cuando el usuario elige una opción correcta, se debe ejecutar el código asociado a dicha opción. Al finalizarla, volverá a mostrar este menú y a pedir otra opción, hasta que el usuario decida volver al menú principal utilizando la opción b.

5.6.1. Import from CSV

Esta opción permite importar información de jugadores almacenados en un **fichero de texto** en formato CSV. Usaremos el carácter coma (',') como separador.

Esta opción se activa cuando el usuario elige la opción 1 del menú de manejo de ficheros. En primer lugar se pedirá al usuario el nombre del fichero donde está almacenada la información mostrando el mensaje:

```
Terminal
Enter filename:
```

El nombre del fichero podrá contener espacios en blanco. No es necesario realizar ninguna comprobación de formato. A continuación se abrirá el fichero de texto y se leerá, añadiendo al programa los jugadores almacenados en el fichero. Si el nombre de fichero está vacío o no se puede abrir el fichero, se emitirá el error ERR_FILE y se volverá al menú de manejo de ficheros.

El formato del fichero será el siguiente: name,team,dorsal,position,ratings (si los hay). Aquí tenemos un ejemplo de la estructura de un fichero de texto:

```
Terminal
Kevin Love,Utah Jazz,11,4,5,18,30,-5,0
Facundo Campazzo,Real Madrid,7,1,35,28
Nikola Jokic,Denver Nuggets,15,5
```

En la última línea del ejemplo, podemos ver que Nikola Jokic no tiene ratings.

Al importar un fichero no se perderán los datos que ya hay en el registro Agency, sino que los jugadores que se lean se irán añadiendo al final del vector players en el mismo orden en el que aparezcan en el fichero, asignándoles un identificador único a cada uno (como si se hubieran creado mediante la opción Add player).

Al añadir los jugadores deberán realizarse las validaciones de los campos name, team, dorsal y position, tal y como se describe en la Sección 5.2. También deberá comprobarse que el nombre del jugador no exista ya en la agencia (mostrando ERR_NAME_EXISTS si existe). Asimismo, deberá comprobarse que las valoraciones (si las hay) sean válidas según se describe en la Sección 5.3. El formato de las valoraciones será siempre correcto, solo habrá que comprobar que los valores estén dentro del rango permitido.

Si algún campo tiene un valor inválido, el jugador no se añadirá a la agencia, se mostrará el mensaje de error correspondiente y se pasará al siguiente jugador del fichero sin realizar las comprobaciones de los campos restantes para ese jugador.

Desarrollado por @prog2ua y publicado bajo licencia
Creative Commons Reconocimiento 4.0 Internacional

- El formato de las líneas del fichero será siempre correcto, pero los valores de los campos podrán ser incorrectos según las reglas descritas.
- El fichero de entrada puede estar vacío. En ese caso, no se importará nada, pero tampoco se mostrará ningún tipo de error.
- Los ficheros CSV son ficheros de texto. Puedes crear tus propios ficheros de ejemplo con cualquier editor de textos (Ej: `joe`, `vi`, `Bloc de notas`).
- El orden de comprobación de errores en la importación de cada jugador será el mismo que en la Sección 5.2.

5.6.2. Export to CSV

Esta funcionalidad, que se activa con la opción 2 del menú de manejo de ficheros, guardará en un **fichero de texto** la información sobre los jugadores. El formato en el que se guardará la información será el mismo que se ha descrito en la sección anterior para la importación, guardando cada jugador en una línea distinta sin incluir sus identificadores (`id`).

En primer lugar se pedirá al usuario el nombre del fichero donde desea almacenar la información, mostrando el siguiente mensaje:

Terminal
```
Enter filename:
```

Se abrirá el fichero de texto para escritura y se guardará en él la información correspondiente, volviendo a continuación al menú de manejo de ficheros. Si el nombre de fichero está vacío o no se puede abrir el fichero se emitirá el error `ERR_FILE` y se volverá al menú de manejo de ficheros.

Si el fichero de salida ya existe, se sobreescribirá su contenido borrando lo que hubiera antes almacenado en él.

5.6.3. Load data

Esta opción permite cargar en el programa todos los datos de la agencia a partir de un **fichero binario**. Los datos que hubiera en ese momento en la estructura `Agency` del programa se borrarán completamente y se sustituirán por la información leída del fichero. Esta funcionalidad se activa con la opción 3 del menú de manejo de ficheros.

En primer lugar se preguntará al usuario si está seguro de la operación, advirtiéndole de que todos los datos del programa se borrarán:

Terminal
```
All data will be erased. Continue? [y/n]:
```

Si el usuario introduce `Y` o `y`, se procederá a pedir el nombre del fichero a cargar. Si el usuario introduce `N` o `n`, se volverá al menú de manejo de ficheros sin llevar a cabo ninguna acción. En caso de que el usuario introduzca cualquier otro carácter o una cadena vacía, se volverá a mostrar el mismo mensaje solicitando la confirmación.

Al continuar se pedirá al usuario el nombre del fichero binario donde está almacenada la información mostrando el mensaje:

Terminal
```
Enter filename:
```

Desarrollado por @prog2ua y publicado bajo licencia
Creative Commons Reconocimiento 4.0 Internacional

Si el nombre de fichero está vacío o el fichero no se pudiera abrir se emitirá el error `ERR_FILE` y se volverá al menú sin borrar los datos de la agencia. Si el fichero se abre correctamente se procederá con el borrado de los datos actuales de la agencia y con la lectura de los nuevos datos desde el fichero.

La siguiente imagen muestra un ejemplo de cómo podría estar estructurado un fichero binario (es sólo un dibujo para aclarar la estructura; recuerda que el fichero en realidad sólo contiene ceros y unos):

BinAgency	BinPlayer	Ratings	BinPlayer	Ratings
name	id	Game 1	id	Game 1
ProSport Agency	1	5	2	35
nextId	name	Game 2	name	Game 2
3	Kevin Love	18	Facundo Campazzo	28
	Team	Game 3	Team	
	Utah Jazz	30	Real Madrid	
	Dorsal	Game 4	Dorsal	
	11	-5	7	
	Position	Game 5	Position	
	4	0	1	
	numRatings		numRatings	
	5		2	

- En primer lugar aparecería en el fichero un registro de tipo `BinAgency` con el nombre (`ProSport Agency`) y el número del siguiente identificador (`3`).
- A continuación aparecería un registro `BinPlayer` representando el primer jugador, cuyo `id` es 1 y nombre `Kevin Love`, seguido del resto de campos.
- El último campo, `numRatings`, nos indica que a continuación vienen `5` valoraciones que habría que ir leyendo: partido 1 valoración de `5`, partido 2 valoración de `18`,... hasta el partido 5 con valoración de `0`.
- Después aparecería un nuevo registro de tipo `BinPlayer` representando el segundo jugador almacenado en el fichero, con `id` igual a `2` y nombre `Facundo Campazzo`.
- Tras ello, se leerían los sucesivos `ratings` (2 en este caso) y con ellos se completaría toda la información almacenada en este fichero binario.

- Se asume que todos los datos almacenados en el fichero binario son correctos, por lo que no es necesario hacer ninguna comprobación adicional.
- En el fichero binario se almacena tanto el identificador de cada jugador (`id`) como el identificador que deberá asignarse al siguiente jugador que se añada (`nextId`).

5.6.4. Save data

Esta opción permite hacer una copia de seguridad de toda la agencia en un **fichero binario**. Se activa con la opción 4 del menú de manejo de ficheros. En primer lugar, se pedirá al usuario el nombre del fichero binario donde se almacenará la información:

Terminal
```
Enter filename:
```

Desarrollado por @prog2ua y publicado bajo licencia
Creative Commons Reconocimiento 4.0 Internacional

Si el nombre de fichero está vacío o el fichero no se puede abrir, se emitirá el error `ERR_FILE` y se volverá al menú de manejo de ficheros. Si el fichero ya existe, se sobrescribirá todo su contenido borrando lo que hubiera antes almacenado en él. El formato de salida será exactamente el mismo que se ha descrito en el apartado anterior para la opción `Load data` (Sección 5.6.3), por lo que se almacenará:

- Un registro `BinAgency` con la información de la agencia.
- Para cada jugador: (i) un registro `BinPlayer` con la información del jugador, donde `numRatings` contendrá el número de valoraciones que tiene el jugador; (ii) a continuación, `numRatings` valores enteros con las valoraciones del jugador.

- Ten en cuenta que puede que tengas que recortar algunos campo de texto al guardarlos en el fichero binario(`name` y `team` de jugador y `name` de la agencia). Por ejemplo, podrías tener un registro `Player` con un `string` de 60 caracteres almacenada en el campo `name`. Sin embargo, en el fichero binario tienes que guardar una estructura de tipo `BinPlayer` cuyo campo `name` tiene un tamaño máximo de 50. En este caso, tendrías que recortar el nombre original y almacenar sólo los primeros 49 caracteres en fichero (el máximo admitido, ya que tienes que dejar un espacio para incluir el `'\0'` final).
- Asumiremos que al recortar el campo `name` y el campo `team` del jugador, así como el campo `name` de la agencia, los nombres resultantes seguirán siendo válidos. No hace falta que lo compruebes.

6. Argumentos del programa

En esta práctica el programa debe permitir al usuario indicar algunas acciones a realizar mediante el paso de argumentos por línea de comando. Deberás utilizar los parámetros `int argc` y `char *argv[]` de la función `main` para poder gestionarlos. El programa admitirá los siguientes argumentos:

- `-i <fichero texto>`. Permite importar jugadores de un fichero de texto, cuyo nombre deberá indicarse tras el argumento `-i`, comportándose igual que si el usuario seleccionara la opción `Import from CSV` del menú de manejo de ficheros, también a la hora de tratar los errores si alguno de los datos leídos es incorrecto. Por ejemplo, si se lee del fichero un jugador cuyo nombre contiene caracteres no válidos, se emitirá el error `ERR_NAME`, se ignorarán los datos de ese jugador y se seguirá procesando el resto del fichero. Si no se puede abrir el fichero se emitirá el error `ERR_FILE` y el programa se iniciará con normalidad, sin haber cargado datos adicionales. El siguiente ejemplo importaría los datos almacenados en el fichero de texto `players.csv`:

  Terminal
  ```
  $ prac2 -i players.csv
  ```

- `-l <fichero binario>`. Permite cargar información de la agencia desde un fichero binario cuyo nombre deberá indicarse después del argumento `-l`, comportándose igual que si el usuario seleccionara la opción `Load data` del menú de manejo de ficheros. En este caso no deberá pedirse confirmación al usuario para cargar los datos desde el fichero. Si no se puede abrir el fichero, se emitirá el error `ERR_FILE` y el programa se iniciará con normalidad, sin haber cargado datos adicionales. El siguiente ejemplo cargaría los datos almacenados en el fichero binario `agency.bin`:

  Terminal
  ```
  $ prac2 -l agency.bin
  ```

Los dos argumentos son opcionales y pueden no aparecer en la llamada al programa. Un determinado argumento sólo puede aparecer una vez en la llamada, de lo contrario se considerará que los argumentos son erróneos.

Desarrollado por @prog2ua y publicado bajo licencia
Creative Commons Reconocimiento 4.0 Internacional

Los ficheros binarios no tienen por qué tener necesariamente la extensión `.bin` y los de texto no tienen por qué tener la extensión `.csv`. Utilizamos estas extensiones en los ejemplos para que quede más claro con qué tipo de fichero estamos trabajando.

Los dos argumentos pueden aparecer al mismo tiempo en una llamada al programa y además en cualquier orden. Independientemente del orden en el que aparezcan en la llamada, el orden en el que se procesarán los argumentos será el siguiente:

1. En primer lugar se procesará la opción `-l` para cargar datos de fichero binario.
2. En segundo lugar se ejecutará la opción `-i` para importar datos desde un fichero de texto.

En el siguiente ejemplo, en primer lugar se cargarán los datos del programa almacenados en el fichero `agency.bin` y, a continuación se incorporarán los jugadores almacenados en el fichero `players.csv`:

Terminal
```
$ prac2 -i players.csv -l agency.bin
```

Otro ejemplo, cuyos parámetros son válidos, sería el siguiente:

Terminal
```
$ prac2 -l -l -i -i
```

Aunque a primera vista los parámetros pudieran parecer incorrectos, en realidad no lo son. El programa recibe un primer parámetro `-l` que va seguido de un fichero que se llama `-l`. A continuación recibe un parámetro `-i` que va seguido de un fichero que se llama `-i`. Esta secuencia de parámetros es, por tanto, correcta.

En caso de que se produzca un error en los argumentos de entrada (por ejemplo, que se introduzca una opción que no existe), se deberá emitir el error `ERR_ARGS` y se finalizará el programa sin llegar a mostrar el menú principal. Si todos los argumentos son correctos, se procesarán las opciones introducidas por el usuario y después se mostrará el menú principal de programa de la forma habitual. Los argumentos pueden ser erróneos si se da alguna de las siguientes circunstancias:

- Alguna de las opciones aparece repetida. Por ejemplo:

  Terminal
  ```
  $ prac2 -l agency.bin -l moredata.bin
  ```

- A las opciones `-l` o `-i` les falta el argumento de detrás con el nombre del fichero. Por ejemplo:

  Terminal
  ```
  $ prac2 -l agency.bin -i
  ```

- Aparece un argumento que no es ninguno de los permitidos. Por ejemplo:

  Terminal
  ```
  $ prac2 -n players.csv
  ```

Desarrollado por @prog2ua y publicado bajo licencia
Creative Commons Reconocimiento 4.0 Internacional